

UNIVERSIDAD NACIONAL
AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA



LABORATORIO 05

MEXI BANCO CON DOCKER COMPOSE

Reporte Mexi Banco v05.1

DATOS ACADÉMICOS

Alumna	Estrella de Maria Mendoza Camacho
No. de cuenta	321132927
Profesor	Oscar Manuel Ruiz Hurtado
Grupo	2
Semestre	2027-1

1 Preparación y ejecución del proyecto

Se trabajó con la versión `v05.1` de *Mexi Banco*, una aplicación bancaria de prueba desarrollada en Java. Para evitar afectar la rama usada en prácticas posteriores, se preparó una carpeta independiente para S05. Se utilizaron Docker Desktop, Docker Compose, Git y Postman.

```
PS C:\Users\Billie Jean\mexi-banco-s05> docker compose ps
NAME                  IMAGE                COMMAND                  SERVICE   CREATED              STATUS                        PORTS
mexi-banco-s05-app-1  mexi-banco-s05-app   "java -jar app.jar"      app       About a minute ago   Up About a minute (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-s05-db-1   postgres:17-alpine   "docker-entrypoint.s…"   db        About a minute ago   Up About a minute (healthy)   5432/tcp
PS C:\Users\Billie Jean\mexi-banco-s05>
```

Figura 1: Servicios `app` y `db` iniciados correctamente y con estado `healthy`.

2 Pruebas funcionales con Postman

2.1 Preparación de la colección y salud de la aplicación

Se instaló Postman de escritorio y se importó la colección *Mexi Banco v05.1* suministrada para el laboratorio. La disponibilidad del banco se comprobó mediante el endpoint `/actuator/health`; la aplicación respondió `200 OK` con `status: UP`.

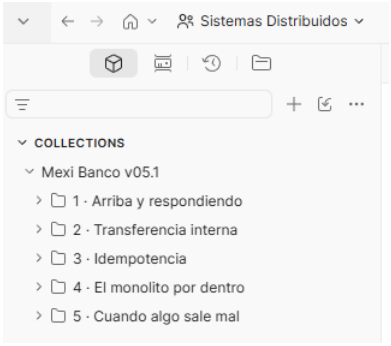


Figura 2: Colección Mexi Banco v05.1.

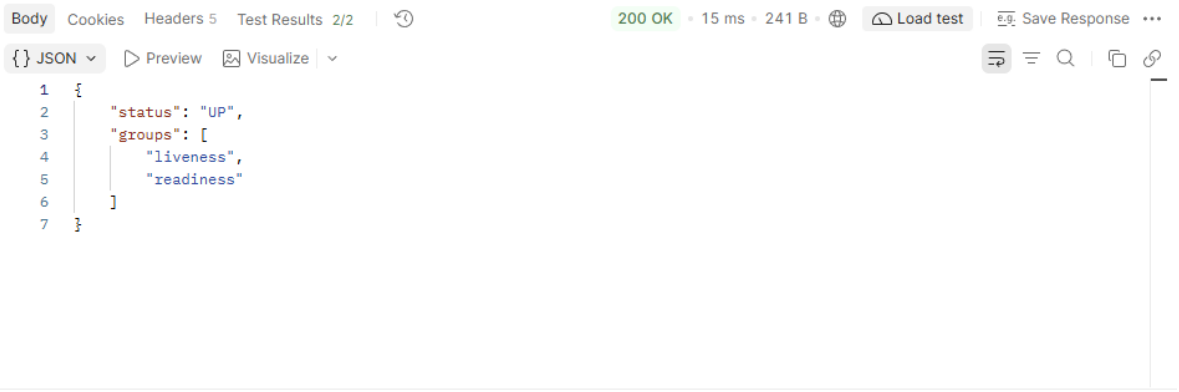


Figura 3: Verificación correcta de salud: respuesta 200 OK, estado UP y dos pruebas aprobadas.

2.2 Apertura y consulta de cuentas

Se creó la cuenta de Ana con un saldo inicial de $1,000 y la de Beto con $500. El servidor respondió `201 Created` en ambos casos; después se consultó la cuenta de Ana, que devolvió `200 OK` y su saldo inicial.

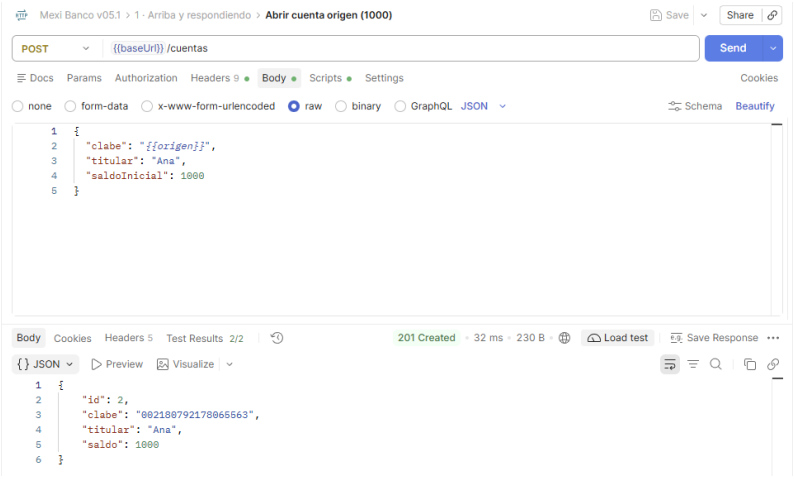


Figura 4: Creación de la cuenta origen a nombre de Ana con saldo inicial de $1,000.

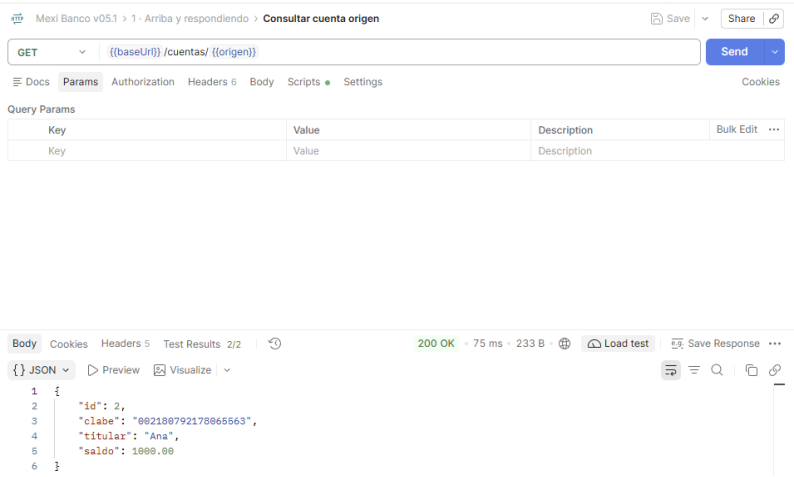


Figura 5: Consulta de la cuenta de Ana y verificación del saldo inicial.

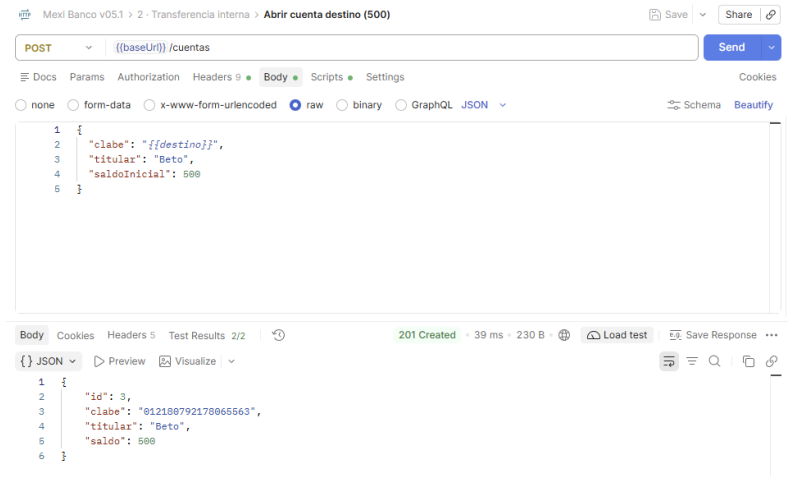


Figura 6: Creación de la cuenta destino a nombre de Beto con saldo inicial de $500.

2.3 Transferencia interna

Se transfirieron $200 desde la cuenta de Ana hacia la de Beto. La operación quedó registrada con un identificador de transferencia. Se consultaron ambas cuentas y se confirmó que el saldo de Ana pasó a $800 y el de Beto a $700.

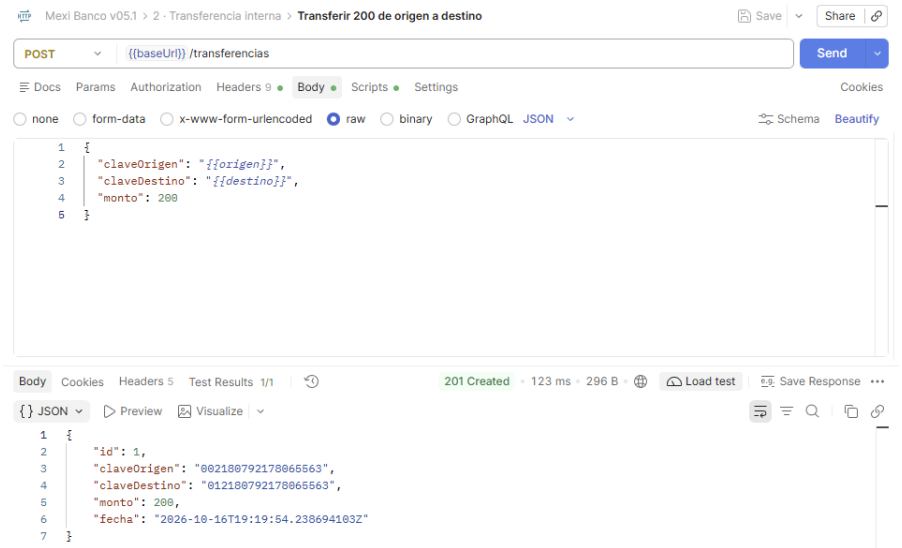


Figura 7: Transferencia interna de $200 registrada con respuesta 201 Created.

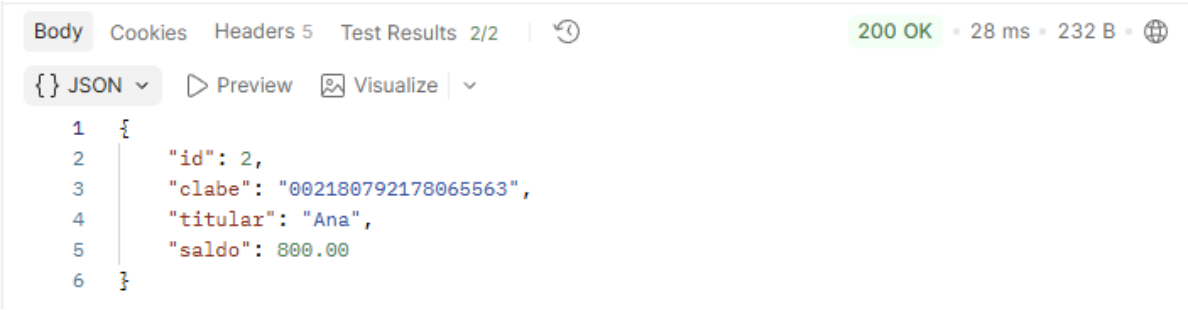


Figura 8: Saldo de Ana después del cargo: $800.



Figura 9: Saldo de Beto después del abono: $700.

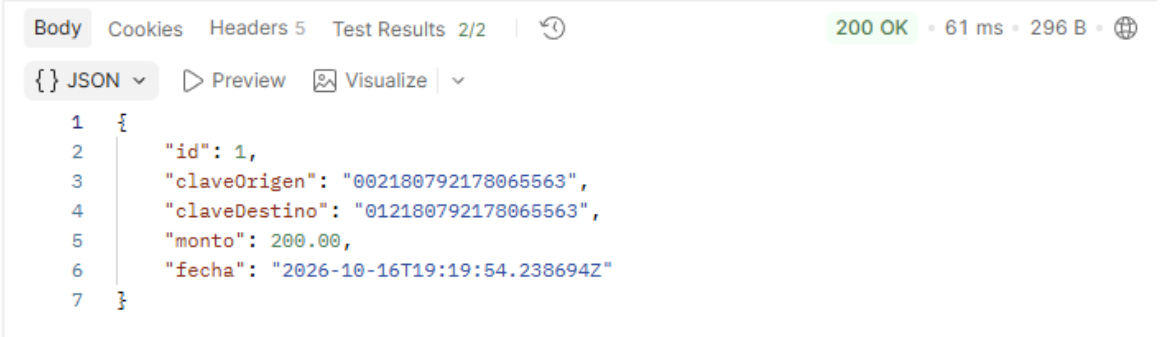


Figura 10: Consulta posterior de la transferencia registrada, con respuesta 200 OK.

3 Idempotencia y trazabilidad

3.1 Envío SPEI y reintento

Se realizó un envío SPEI de $50 desde la cuenta de Ana. Luego se repitió la solicitud con la misma llave de idempotencia. El banco devolvió el mismo identificador del envío y el saldo de Ana permaneció en $750, por lo que no se efectuó un segundo cargo.

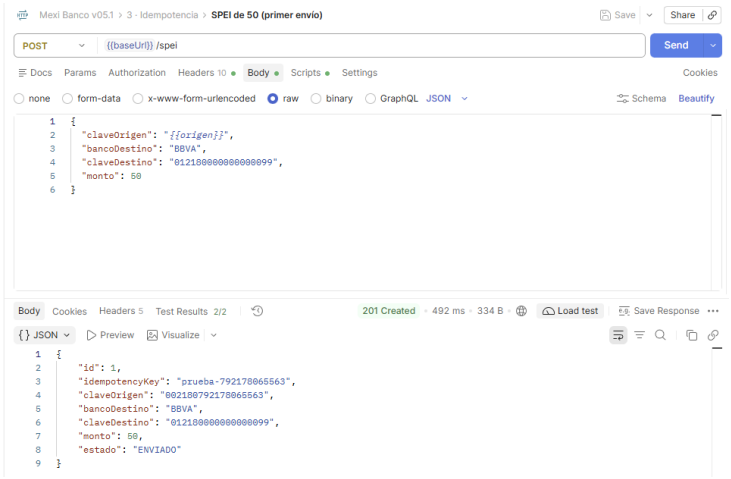


Figura 11: Primer envío SPEI de $50, con resultado ENVIADO.

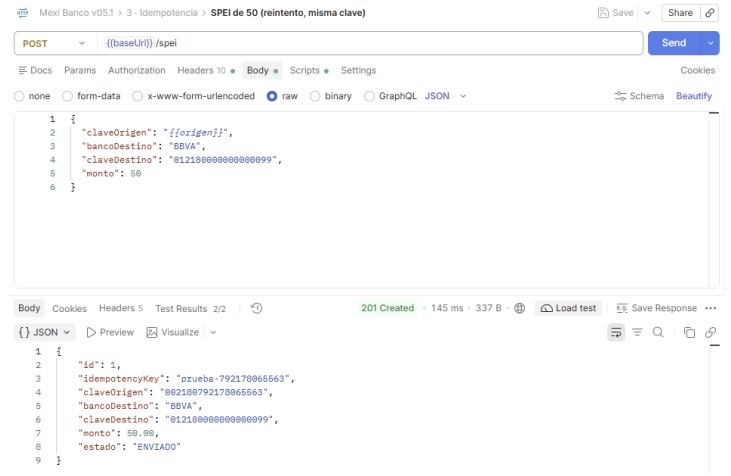


Figura 12: Reintento SPEI, se recuperó la operación original.

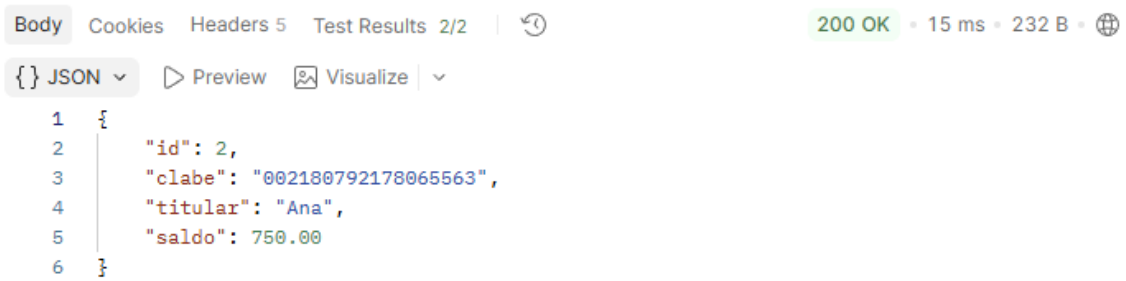


Figura 13: Saldo de Ana en $750, sin descuentos adicionales por el reintento.

3.2 Movimientos y notificaciones

Se consultó el historial de la cuenta origen. Se observaron el depósito inicial, la transferencia de $200 y el SPEI de $50. También se consultaron los avisos: apareció uno por la transferencia y otro por el SPEI, sin notificación adicional por el reintento.

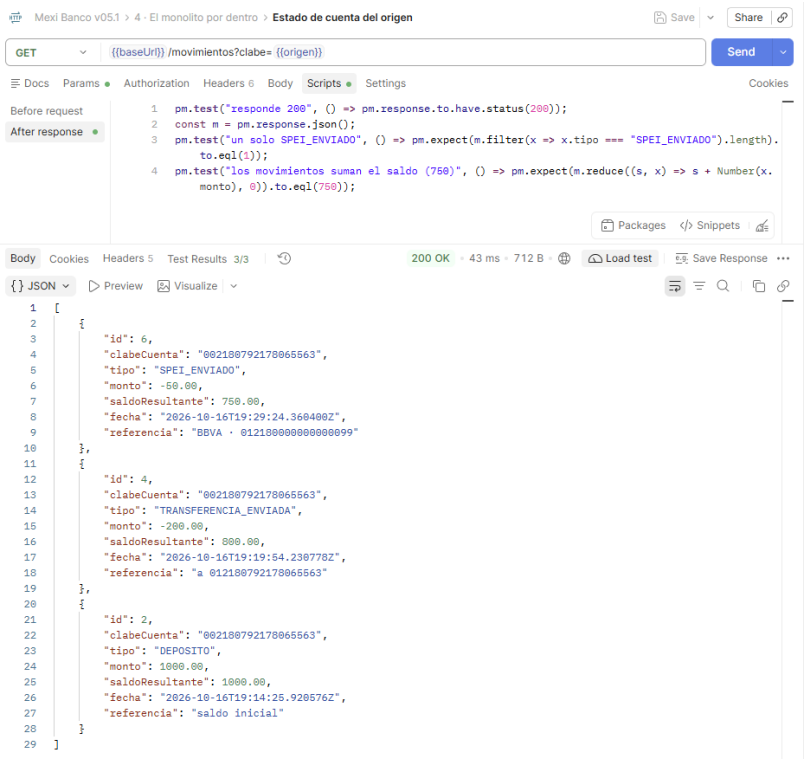


Figura 14: Estado de cuenta con tres movimientos y saldo resultante de $750.

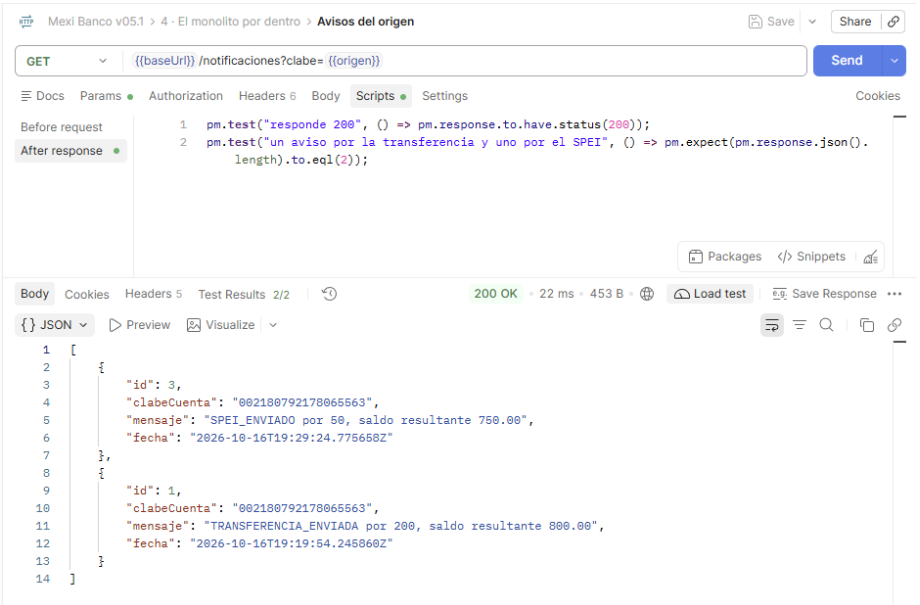


Figura 15: Notificaciones generadas por la transferencia interna y el envío SPEI.

4 Manejo de errores

La colección permite enviar solicitudes inválidas a propósito. En estos casos, recibir un código HTTP de error es el resultado esperado: muestra que la aplicación valida las entradas y las reglas de negocio.

Caso probado	HTTP	Interpretación
CLABE inexistente	404	Cuenta no encontrada
Cuenta duplicada	409	Conflicto de datos
Solicitud de cuenta inválida	400	Datos incorrectos
Saldo insuficiente	422	Operación no permitida
Origen y destino iguales	422	Regla de negocio incumplida
Solicitud SPEI inválida	400	Petición incorrecta
Consulta final de saldo	200	El saldo siguió en $750

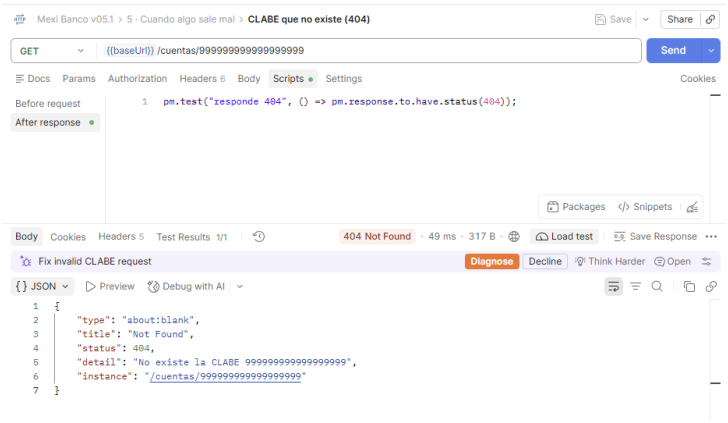


Figura 16: Error 404 al consultar una CLABE inexistente.

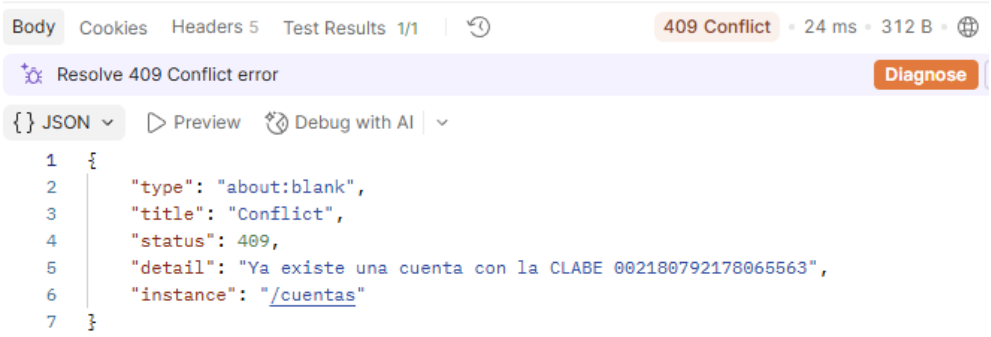


Figura 17: Error 409 al intentar registrar una cuenta que ya existía.

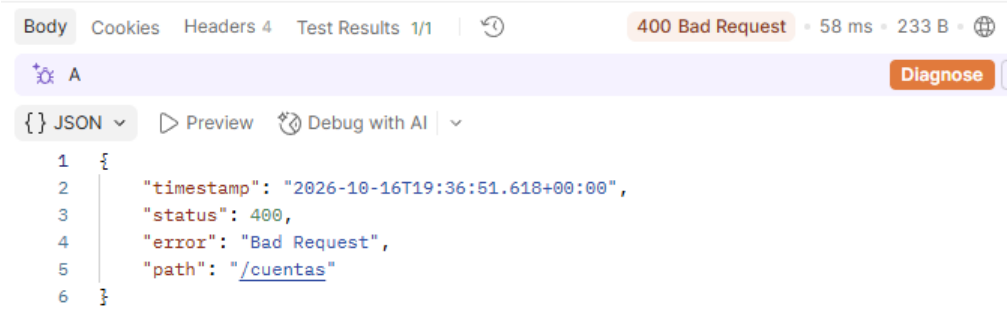


Figura 18: Error 400 por una solicitud con datos inválidos.

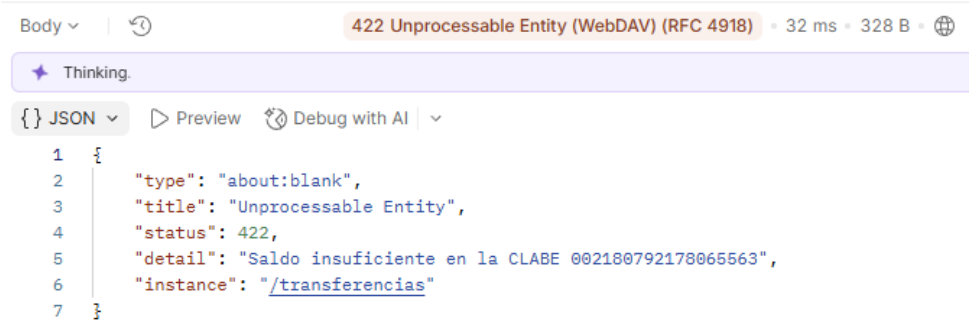


Figura 19: Error 422 al intentar transferir sin fondos suficientes.

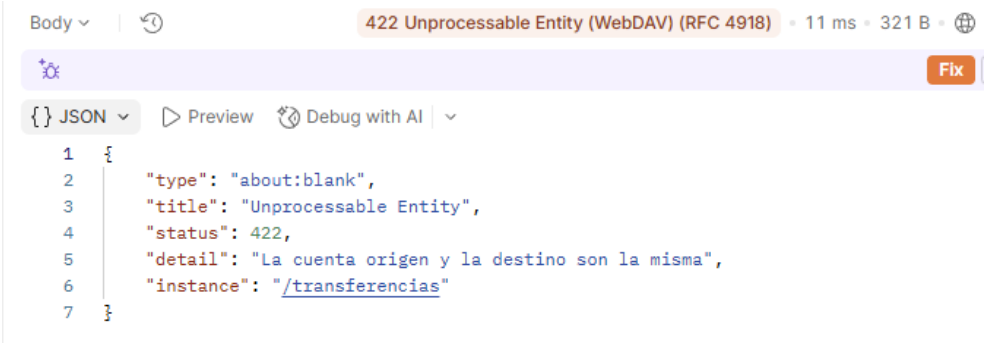


Figura 20: Error 422 al intentar transferir a la misma cuenta.

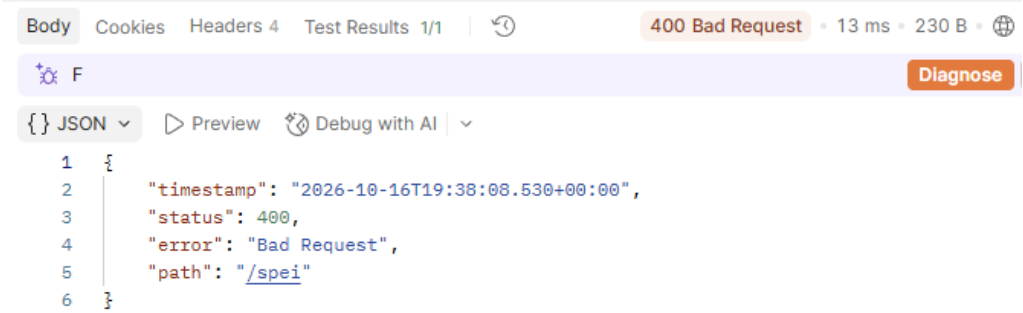


Figura 21: Error 400 al enviar una petición SPEI inválida.

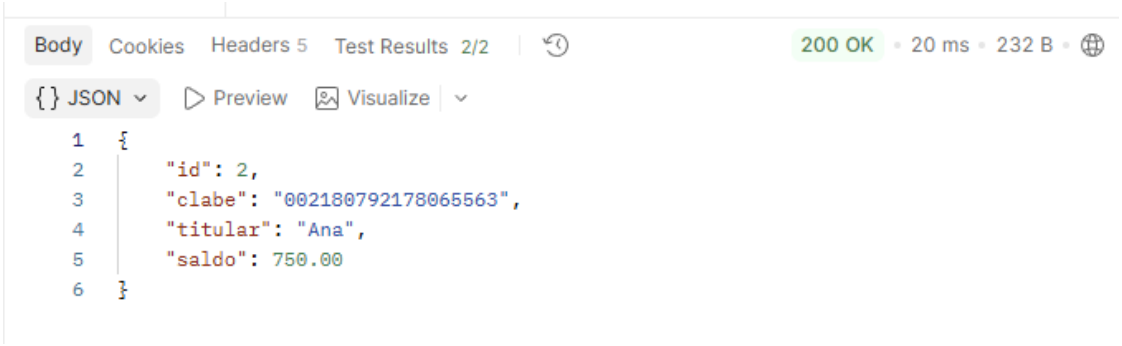


Figura 22: Comprobación posterior: la cuenta de Ana conservó $750.

5 Persistencia de datos en Docker

5.1 Reinicio sin borrar el volumen

Se ejecutó `docker compose down` y posteriormente `docker compose up -d`. Los contenedores fueron recreados; al consultar la cuenta de Ana, el saldo de $750 seguía almacenado. Esto indica que el volumen de PostgreSQL no fue eliminado.

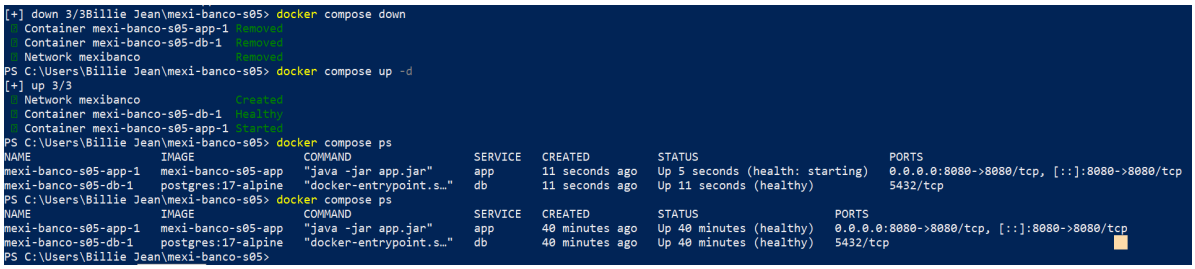


Figura 23: Apagado y nuevo arranque de los servicios; la aplicación todavía iniciaba en la primera consulta de estado.

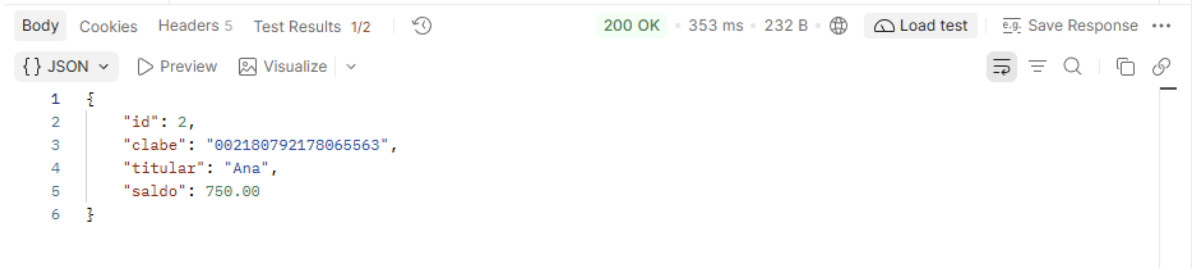


Figura 24: Consulta después del reinicio: los $750 de Ana permanecieron guardados.

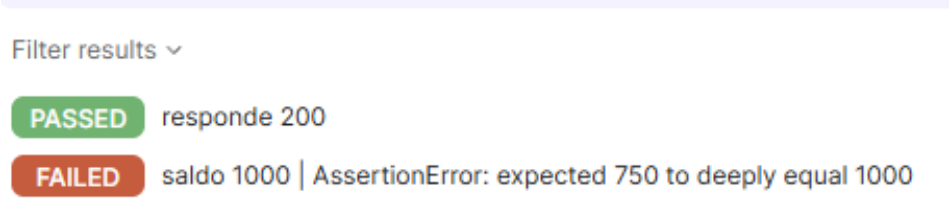


Figura 25: Una prueba esperaba el saldo inicial de $1,000; falló por ese valor fijo.

5.2 Reinicio eliminando el volumen

Después se utilizó `docker compose down -v`. Este comando retiró los contenedores y el volumen nombrado. Al volver a levantar la aplicación, PostgreSQL creó un volumen nuevo y la cuenta de Ana dejó de existir. Por eso, la consulta devolvió `404 Not Found`.

```
PS C:\Users\Billie Jean\mexi-banco-s05> docker compose down -v
[+] down 4/4
 ✔ Container mexi-banco-s05-app-1        Removed                                                    0.4s
 ✔ Container mexi-banco-s05-db-1         Removed                                                    0.4s
 ✔ Network mexibanco                     Removed                                                    0.3s
 ✔ Volume mexi-banco-s05_mexibanco_datos Removed                                                    0.0s
Users\Billie Jean\mexi-banco-s05>
>> docker compose up -d
[+] up 4/4
 ✔ Volume mexi-banco-s05_mexibanco_datos Created                                                    0.1s
 ✔ Network mexibanco                     Created                                                    0.1s
 ✔ Container mexi-banco-s05-db-1         Healthy                                                    6.4s
 ✔ Container mexi-banco-s05-app-1        Started                                                    6.5s
Users\Billie Jean\mexi-banco-s05> docker compose ps
NAME                  IMAGE                 COMMAND                  SERVICE   CREATED          STATUS                            PORTS
mexi-banco-s05-app-1  mexi-banco-s05-app    "java -jar app.jar"      app       12 seconds ago   Up 5 seconds (health: starting)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-s05-db-1   postgres:17-alpine    "docker-entrypoint.s…"   db        12 seconds ago   Up 11 seconds (healthy)           5432/tcp
PS C:\Users\Billie Jean\mexi-banco-s05> docker compose ps
NAME                  IMAGE                 COMMAND                  SERVICE   CREATED          STATUS                            PORTS
mexi-banco-s05-app-1  mexi-banco-s05-app    "java -jar app.jar"      app       19 seconds ago   Up 12 seconds (health: starting)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-s05-db-1   postgres:17-alpine    "docker-entrypoint.s…"   db        19 seconds ago   Up 17 seconds (healthy)           5432/tcp
PS C:\Users\Billie Jean\mexi-banco-s05> docker compose ps
NAME                  IMAGE                 COMMAND                  SERVICE   CREATED          STATUS                    PORTS
mexi-banco-s05-app-1  mexi-banco-s05-app    "java -jar app.jar"      app       34 seconds ago   Up 27 seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-s05-db-1   postgres:17-alpine    "docker-entrypoint.s…"   db        34 seconds ago   Up 33 seconds (healthy)   5432/tcp
PS C:\Users\Billie Jean\mexi-banco-s05>
```

Figura 26: Eliminación y recreación de contenedores, red y volumen mediante Docker Compose.

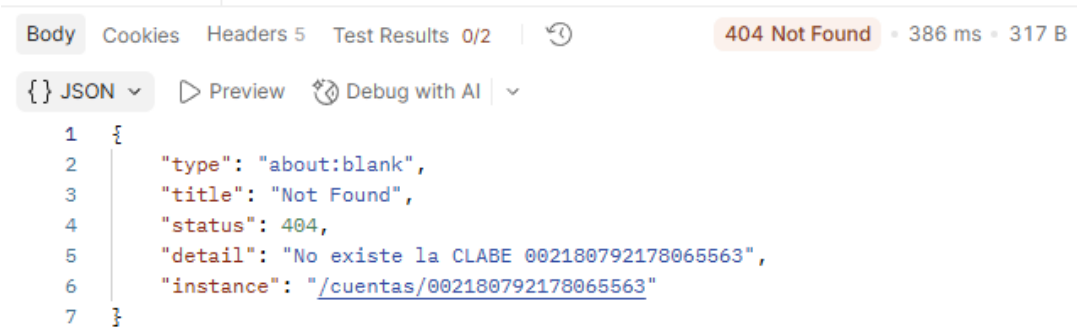


Figura 27: Consulta de la antigua cuenta de Ana: error 404 tras eliminar el volumen de datos.

6 Revisión del código del monolito

Se exploraron las carpetas `cuenta`, `movimiento`, `transferencia`, `notificacion` y `spei`. Cada módulo separa las responsabilidades en controladores, servicios, repositorios y entidades. En particular, `TransferenciaService` coordina la validación, el cargo, el abono, los movimientos y las notificaciones.

La anotación `@Transactional` permite que las operaciones de base de datos de la transferencia interna se manejen como una sola transacción local: ante una falla que provoque *rollback*, sus cambios se revierten. Este enfoque es posible porque, en S05, los módulos funcionan como un monolito con una base de datos compartida.

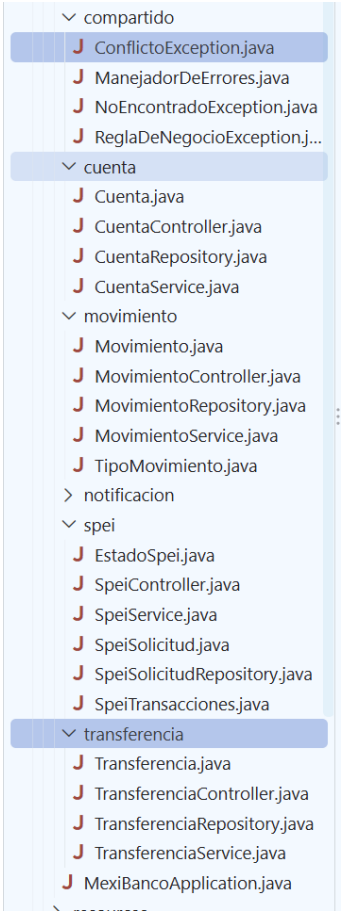


Figura 28: Organización de paquetes Java y sus capas principales.

```java
@Transactional
public Transferencia transferir(String claveOrigen, String claveDestino, BigDecimal monto) {
    if (claveOrigen.equals(claveDestino)) {
        throw new ReglaDeNegocioException("La cuenta origen y la destino son la misma");
    }
    // Primero que existan las dos, luego el saldo: el mismo orden de errores que antes del refactor
    cuentas.consultar(claveOrigen);
    cuentas.consultar(claveDestino);

    Cuenta origen = cuentas.cargar(claveOrigen, monto);
    Cuenta destino = cuentas.abonar(claveDestino, monto);

    Movimiento cargo = movimientos.registrar(origen.getClabe(), TipoMovimiento.TRANSFERENCIA_ENVIADA,
        monto.negate(), origen.getSaldo(), "a " + destino.getClabe());
    Movimiento abono = movimientos.registrar(destino.getClabe(), TipoMovimiento.TRANSFERENCIA_RECIBIDA,
        monto, destino.getSaldo(), "de " + origen.getClabe());

    Transferencia transferencia = transferencias.save(new Transferencia(claveOrigen, claveDestino, monto));

    notificaciones.notificar(cargo);
    notificaciones.notificar(abono);
    return transferencia;
}
```

Figura 29: Método `transferir()` y anotación `@Transactional` dentro de `TransferenciaService`.

7 Configuración de Docker Compose y Dockerfile

El archivo `docker-compose.yml` declara dos servicios: `db`, con PostgreSQL 17, y `app`, con la aplicación Java. La aplicación utiliza `DB_HOST=db`; Docker resuelve ese nombre dentro de la red del proyecto. La condición `service_healthy` evita que la aplicación arranque antes de que

la base de datos esté disponible. El volumen `mexibanco_datos` almacena la información de PostgreSQL.

docker-compose.yml

```yaml
# Con un solo comando: docker compose up --build
# Lo que van a ver correr (S05, "de compose a orquestacion"):
#   - `app` resuelve `db` por nombre (DNS embebido de Compose), sin ninguna IP en ningun lado.
#   - `app` espera a que `db` este listo de verdad (depends_on + healthcheck), no solo que exista.
#   - Las variables de entorno son la configuracion (factor III, ya visto en S04).
#   - `db` persiste en un volumen nombrado: si tiran el contenedor y lo vuelven a subir, los
#     datos siguen ahi; si tiran el volumen, no.
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5

networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

Figura 30: Archivo `docker-compose.yml`: servicios, variables de entorno, dependencias, red, comprobaciones de salud y volumen.

El `Dockerfile` utiliza una construcción de dos etapas. La primera incluye JDK 21 y Maven para generar el archivo `.jar`; la segunda utiliza JRE 21 para ejecutar el archivo compilado. Durante la práctica se añadió un comando que normaliza los saltos de línea de `mvnw` y habilita su ejecución en Linux, resolviendo el error de compilación observado inicialmente.

Dockerfile

```dockerfile
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN sed -i 's/\r$//' mvnw && chmod +x mvnw
RUN ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]

```

Figura 31: Dockerfile multietapa con la corrección aplicada al script `mvnw`.

8 Conclusiones

Se logró ejecutar Mexi Banco con Docker Compose y comprobar su funcionamiento mediante Postman. Se verificaron la creación y consulta de cuentas, una transferencia interna, un SPEI simulado y el comportamiento idempotente de los reintentos. Las respuestas de error mostraron las validaciones del sistema sin afectar indebidamente los saldos.

La comparación entre `down` y `down -v` permitió comprobar la función de los volúmenes en la persistencia. Finalmente, al revisar los módulos Java, Docker Compose y el Dockerfile, se comprendió cómo un monolito integra su lógica bancaria, se comunica con PostgreSQL y se construye para ejecutarse en contenedores.